# Project Tutorial

## Build a DApp

# DApp Architecture Overview

- **User & Wallet Layer**
  Users interact via browsers (Chrome/Firefox) with MetaMask/WalletConnect for secure identity verification and cryptographic signing of transactions.
- **Frontend Interface (Client-Side)**
  Built with JavaScript & React; renders dynamic UI
- **Communication Layer: Ethers library**
- A JavaScript library acting as the bridge; connects the frontend to the blockchain, reads contract state.
- **Smart Contract Layer (Sepolia Testnet)**
  Solidity-written logic deployed on Ethereum's Sepolia

# Contract.js

## Key Initialization Concepts

- **Contract Address:** The unique identifier of your smart contract deployed on the blockchain network.

- **ABI (Application Binary Interface):** A JSON schema that defines the contract's functions, events, and data types, generated during compilation.

- **Testnet**

- **Example**

```
export const CONTRACT_ADDRESS = XXX

export const TICKET_ABI = [
  "function mint() returns (uint256 tokenId)",
  "function hasMinted(address account) view returns (bool)",
  "event Mint(address indexed minter, uint256 indexed tokenId)",
];


export const SEPOLIA_CHAIN_ID = "0xaa36a7";

export const SEPOLIA_EXPLORER_URL = "https://sepolia.etherscan.io";
```

# Solidity

- **Event:** Transaction Log
- **external:** Can only be called externally.

```
contract TicketNFT is ERC721 {
    uint256 private _nextTokenId = 1;

    mapping(address => bool) public hasMinted;
    event Mint(address indexed minter, uint256 indexed tokenId);

    constructor() ERC721("Ticket NFT", "TICKET") {}

    function mint() external returns (uint256 tokenId) {

        require(!hasMinted[msg.sender], "One ticket per wallet");
        tokenId = _nextTokenId;
        _nextTokenId += 1;
        hasMinted[msg.sender] = true;
        _safeMint(msg.sender, tokenId);
        emit Mint(msg.sender, tokenId);
    }
}
```

# Ethers.js

01. Ensure you have installed Node.js LTS

02. Run npm install to install all dependencies.

Provider read

Signer write

Contract

# Ethers interaction logic

## 01. Get provider

```
function getProvider() {
    if (!window.ethereum) throw new Error("Install MetaMask to continue");
    return new BrowserProvider(window.ethereum);
  }
```

# Ethers interaction logic

## 02. Ensure Testnet

```
async function ensureSepolia() {
    const currentChainId = await window.ethereum.request({ method: "eth_chainId" });
    if (currentChainId === SEPOLIA_CHAIN_ID) return;
    await window.ethereum.request({
      method: "wallet_switchEthereumChain",
      params: [{ chainId: SEPOLIA_CHAIN_ID }],
    });
  }
```

# Ethers interaction logic

## 03. Connect Wallet

```
async function connectWallet() {
    const provider = getProvider();
    const accounts = await provider.send("eth_requestAccounts", []);
    await ensureSepolia();
    setWalletAddr(accounts[0]);

}
```

# Ethers interaction logic

## 04. Mint

```
async function mint() {

    await ensureSepolia();
    const provider = getProvider();
    const signer = await provider.getSigner();
    const contract = new Contract(CONTRACT_ADDRESS, TICKET_ABI, signer);
    const transaction = await contract.mint();
    setTransactionId(transaction.hash);
    setHintText("Transaction submitted. Waiting for confirmation...");
    await transaction.wait();
    setClaimedNft(true);
    setHintText("Ticket minted successfully!");
}
```

# Ethers interaction logic

## 05. Monitoring wallet events

```
useEffect(() => {
  if (!window.ethereum) return undefined;

  const AccountChanged = (accounts) => {
    const nextAccount = accounts[0] || "";
    setWalletAddr(nextAccount);
    setTransactionId("");
    setHintText("");
  };

window.ethereum.on("accountsChanged", AccountChanged);

return () => {
    window.ethereum.removeListener("accountsChanged", AccountChanged);
  };
});
```

# Others

- **CSS file**
- Page beautification, color and layout

- **Main.jsx file**
- Application entry, render the App component

# Example

# Questions

豆包AI生成